05

Production

What a server is, and what the hosting choice decides for you

At some point the app leaves your laptop and runs somewhere users can reach it. The choices at that point look like a list of brand names, and are better read as positions on one axis: how much is decided for you. This chapter gives the axis, the vocabulary that makes hosting conversations followable, and the checks for the claims a deployed app quietly makes — that it can be rolled back, that its backups restore, that someone hears its alerts.

PART

The model

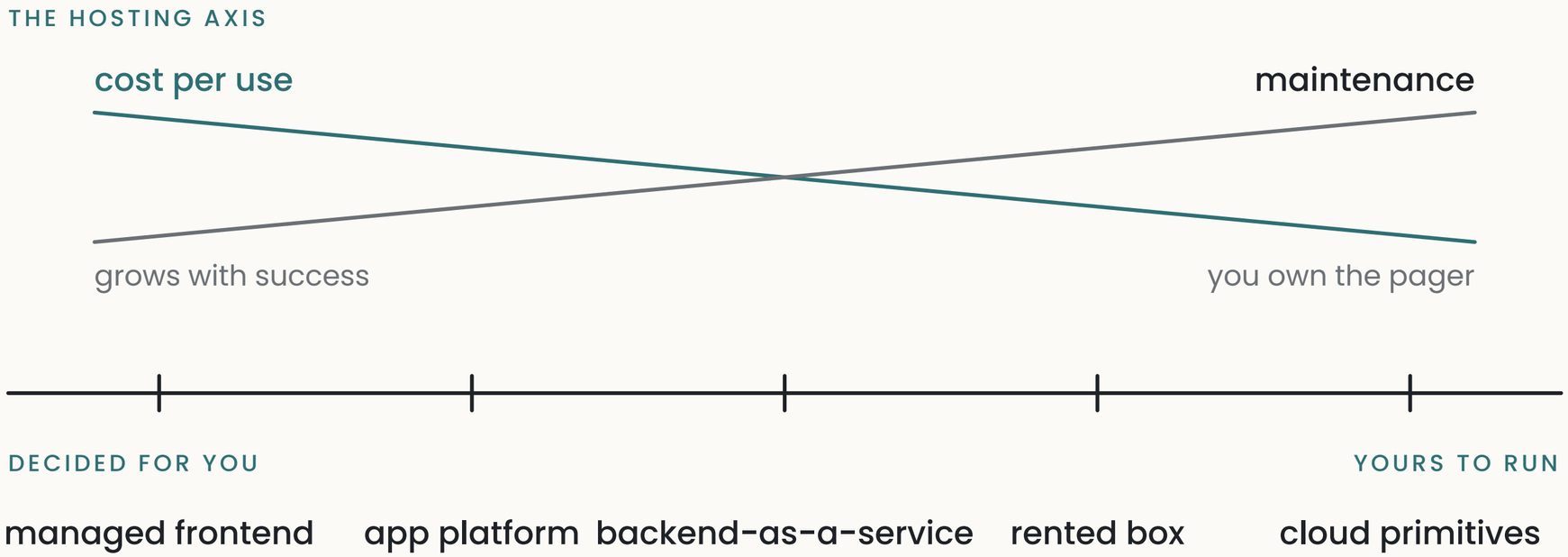
What a server is

A server is a computer — usually rented, sitting in a data centre, running Linux — executing your code as processes, exactly as your laptop does. Three things distinguish it: it stays on, it's reachable at a public address, and somebody has to maintain it — and most hosting decisions come down to who that somebody is.

The hosting axis

Arranged by how much is decided for you:

KIND	WHAT YOU GIVE THEM	WHAT STAYS YOURS
Managed frontend platform	The code; they build, host, and scale it	Almost nothing to operate
App platform (PaaS)	An app to run; they run it	Configuration, scaling choices
Backend-as-a-service	Nothing to host; you use their database, auth, storage	Your schema, policies, client code
Rented box (VPS)	Money	The whole machine: installs, updates, restarts, security
Cloud primitives	Money	Everything, assembled from parts



The left end is fast to start, opinionated, and priced per use, which grows with success. The right end is cheap and flexible, and you carry the maintenance. Neither end is correct in general; the axis exists so that “should we move” is a question about position, trade-offs, and exit cost rather than about brands.

Serverless and long-running

On the left of the axis, code commonly runs **serverless**: your function is started when a request arrives, answers it, and is gone. Three constraints follow directly. A **cold start** — the delay when a request arrives and nothing is running yet.

An **execution limit** — the platform stops functions that run too long, which is why chapter 4's background work needs somewhere else to live. And **statelessness** — nothing survives in memory between requests, so anything worth keeping goes to the database. A **long-running** process — the default on the right of the axis — has none of these constraints and instead needs supervision: something to restart it when it dies.

Environments and deploys

An **environment** is a complete copy of the app with its own configuration and data: local for building, production for users, sometimes staging between them. A **deploy** builds the code and swaps it into an environment. "Works locally" and "works deployed" are different claims because between the two environments the machine, the configuration, the data, and the platform constraints all change — which is most of why things break at deploy time. The rule that follows: when a deploy breaks something that worked locally, roll back first and debug after, rather than fixing forward by pasting errors until it works.

A **rollback** is returning to the previous build. The property worth having is knowing, before deploying, exactly what getting back takes — platforms on the left of the axis usually make it one click, and it's worth confirming rather than assuming.

Names and certificates

DNS is a public lookup table from names to addresses; your domain points at your app because a DNS record says so, and editing the record is how a domain moves. Changes propagate slowly — old answers are cached for hours. **TLS** is what makes a site `https`: a certificate proving the server is the one the name points to. Platforms automate issuing and renewing certificates; the recurring failure is an expired one, which takes a site down with an error every visitor sees.

Backups

A backup is a copy of the data that can be restored. Both halves count: the copy must exist on a schedule, and a restore must have been performed — a backup that has never been restored is a copy whose restorability nobody has checked. The restore test belongs on a calendar, since nothing about normal operation will ever exercise it.

Knowing when it’s broken

In production nobody is watching the screen, so a failure is only as visible as the trace it leaves. Three mechanisms cover it at recognition depth. **Logs** are the lines a process writes as it runs, kept by the platform for a while; they’re where “what happened at three in the morning” gets answered, if the code wrote anything. **Error tracking** is a service that collects every unhandled failure with its context and counts them, so a bug that hits one user in a hundred shows up as a number rather than as a complaint. **Alerts** are the rule that turns a count or a log line into a message to a person.

Two things about what gets recorded are worth knowing. A failure that writes nothing leaves no trace, and background work (chapter 4) is where that happens most, since no user is there to complain. And logs are a place data ends up: a request logged in full carries the user’s email, their address, sometimes a password or a key, into a store with its own retention and its own audience — chapter 7’s rule about shared artifacts applies to logs too.

What the user sees when something fails is a separate decision. A platform’s default is often the whole internal error — file paths, table names, the query that failed — which tells the user nothing useful and tells an attacker a good deal. The user gets a plain message and a reference; the details go to the log.

For a small app with a few hundred users, the floor is error tracking plus one alert that reaches your own inbox; having neither is a decision to make on purpose rather than a default to keep.

PART

What goes wrong

1 No rollback path

the only way back from a bad deploy is fixing forward under pressure.

ASK “If the deploy that just went out is bad, what exactly do I do to get back to the previous one, and how long does it take?”

CHECK practice it once on a harmless change.

TELL *deploys that overwrite in place, with no previous build kept.*

2 The untested backup

a copy that has never been proven restorable.

ASK “When did the last successful restore happen, and into what?”

CHECK restore the latest backup somewhere disposable and open the result.

TELL *backups configured, restore never run.*

3 The alert nobody receives

monitoring that reports to an empty room.

ASK “List every alert configured, where each one delivers, and who saw the most recent one fire. Say nobody explicitly.”

CHECK trigger a harmless test alert and see who notices, and how fast.

TELL *alerts pointed at a channel nobody reads or an address nobody owns.*

4 Failure that leaves no trace

something breaks and nothing records that it did.

ASK “For each background job and each external call, say what gets recorded when it fails and where I’d read it. Write nothing where nothing is recorded.”

CHECK make one fail on purpose in a test copy (chapter 0, D1), then go and find the evidence.

TELL *background jobs and external calls with no logging on the failure branch; an app with no error tracking configured.*

5 The error shown in full

internal details delivered to whoever triggered the failure.

ASK “List every place an error reaches the user, and what text it carries. Flag any that includes a file path, a table name, or query text.”

CHECK trigger a failure as a user — a malformed request does it — and read what comes back.

TELL *stack traces, file paths, or query text in an error the browser displays.*

THE DIRECT TEST

First find out whether backups exist at all — on a free tier they often don’t — then restore the most recent one into a disposable copy (chapter 0, D1) and open it. One check settles the most expensive claim on this surface, and doing it on a calendar keeps it settled — the failure it guards against produces no symptom until the day the backup is needed.

PART

Going deeper

These prompts are for your assistant, and each comes with a note on what a good response looks like.

D1 · Map my hosting

GROUNDED EXPLANATION

For each part of my app — frontend, server code, database, files, background jobs — state which platform runs it, where that sits on the decided-for-you axis, and which constraints apply: cold starts, execution limits, statelessness. Then estimate what the whole setup costs at today's usage and at ten times today's.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response places every part somewhere, and the background-jobs row is the one to read closely — on serverless platforms it's where "nothing actually runs this" surfaces.

D2 · The production-claims audit

AUDIT

List everything that has to be true in production for my app to work, that nothing currently checks: environment variables present, scheduled jobs actually scheduled, domains pointing at the right place, certificates renewing, backups running. For each: how would I find out if it stopped being true — an alert, a symptom, or nothing?

WHAT A GOOD RESPONSE LOOKS LIKE

A good response is a table where the last column has some honest "nothing"s. Each "nothing" is a silent failure waiting for its chapter 12 scheduled audit.

D3 · Walk the deploy

WALK THE FAILURE PATH

Walk me through what happens when I deploy: the build, the swap, what users with the app open mid-deploy experience, what happens to requests in flight, and at which point the old version stops being reachable. Then the same walk for a rollback.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response reaches the moment of swap and says what happens on either side of it. If it can't say what mid-deploy users see, that's worth finding out before a deploy that matters.

D4 · The rollback drill

BUILD A TOY

Help me practice a rollback: deploy a trivial visible change, confirm it's live, then roll back to the previous build and confirm that's live again. Time both directions.

WHAT A GOOD RESPONSE LOOKS LIKE

It takes about fifteen minutes, and "we can roll back" becomes something you've done with a duration attached, rather than a setting you believe exists.

WHERE THIS CONNECTS

Chapter 4 · The request lifecycle is where the execution-limit constraint bites — its scheduled-work entry lives on this chapter's model. **Chapter 7 · Secrets and configuration** covers what differs between environments, and its shared-artifact rule reaches logs. **Chapter 10 · Version control** distinguishes rolling back a deploy from reverting code — related moves at different layers.